# Scope and Environments

## Chapter 2

Stats 102A: Introduction to Computational Statistics with R

UCLA


Acknowledgements: Miles Chen and Michael Tsiang

(This chapter is largely drawn from Hadley Wickham's Advanced R, 2019)

Section 1

# Scope

# Scope

**Scoping** is the act of finding the value associated with a name. The **scope** of a variable is the region of the code where the variable is defined.

Consider the following code:

```
y <- 1
g <- function(x){
  y <- 2
  x + y
}
```

**Question**: What is the output of `g(3)`?

# Lexical Scope

R uses lexical scoping: it looks up the values of names based on how a function is defined, not how it is called. R's lexical scoping follows four primary rules:

- Name masking
- Functions versus variables
- A fresh start
- Dynamic lookup

# Name Masking

Names defined inside a function **mask** names defined outside a function.

```
y <- 1
g <- function(x){
  y <- 2
  x + y  # finds y in the current environment
}
g(3)
```

```
## [1] 5
```

If a name isn't defined inside a function, R looks one level up.

# Functions versus variables

In R, functions are ordinary objects. This means the scoping rules described above also apply to functions:

```
g1 <- function(x) x + 1

g2 <- function() {
  g1 <- function(x) x + 100
  g1(10)
}
g2()
```

```
## [1] 110
```

# A fresh start

Every time a function is called a new environment is created to host its execution.

```
g3 <- function() {
  if (!exists("a")) {
    a <- 1
  } else {
    a <<- a + 1
  }
  a
}
g3()
```

```
## [1] 1
```

```
g3()
```

# Dynamic lookup

Lexical scoping determines where, but not when to look for values. R looks for values when the function is run, not when the function is created.

```
g4 <- function() x + 1
x <- 15
g4()
```

```
## [1] 16
```

```
x <- 20
g4()
```

```
## [1] 21
```

Section 2

# Environments

# Introduction

The **environment** is the data structure that powers scoping.

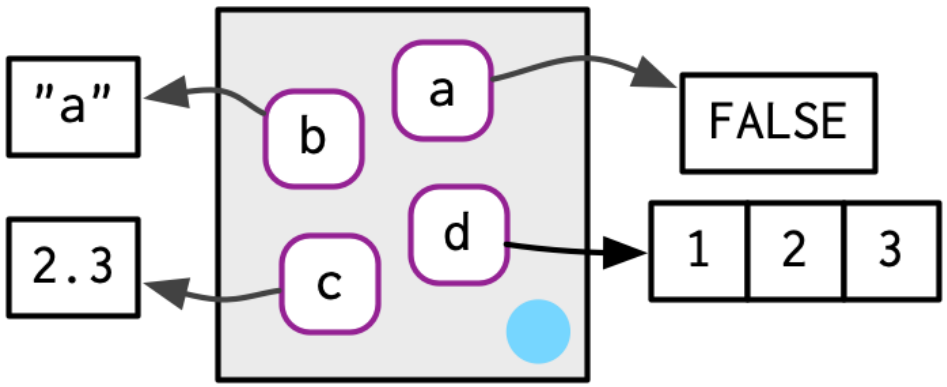
- All objects created and stored in an R session are collectively called the **workspace**, also known the **global environment**.
- The body of a function creates a **local environment**, so objects created inside the function are accessible only within the function. Objects created locally will not appear in the global environment
- Objects in the local environment will take precedence over objects in the global environment. In particular, a global object will be **masked** by a local object with the same name: The object name within the function will reference the local object instead of the global one.

We want to formalize these ideas and give a deeper understanding of scope and environments to help us build more complex programs.

# Environments

The job of an environment is to associate, or **bind**, a set of names to a set of values. You can think of an environment as a bag of names, each name pointing to an object stored elsewhere in memory.

```
library(rlang)
e <- new.env()
e$a <- FALSE
e$b <- "a"
e$c <- 2.3
e$d <- 1:3
```



# Environment Objects

As with everything in R, environments are also objects. The syntax to work with environment objects is similar to lists in that the double bracket `[[` and `$` operator can be used to get and set values.

```
typeof(e)
```

```
## [1] "environment"
```

```
e[["a"]]
```

```
## [1] FALSE
```

```
e$c
```

```
## [1] 2.3
```

Objects inside environments are not ordered, so numeric indices do not work with environments.

# Environment Objects

The `ls()`, `ls.str()` and `env_print()` functions are useful for looking at the objects inside or the structure of the environment.

```
ls.str(e)
```

```
## a :  logi FALSE
## b :  chr "a"
## c :  num 2.3
## d :  int [1:3] 1 2 3
```

```
env_print(e)
```

```
## <environment: 0x12704b070>
## Parent: <environment: global>
## Bindings:
## * a: <lgl>
## * b: <chr>
## * c: <dbl>
```

# Environment Objects

Unlike lists:

- The single square bracket `[` does not work for environments.
- Setting an object to `NULL` does not remove the object.
- Removing objects can be done using the `rm()` function.

```
e$d <- NULL
ls(e)
```

```
## [1] "a" "b" "c" "d"
```

```
rm(d,envir=e)
ls(e)
```

```
## [1] "a" "b" "c"
```

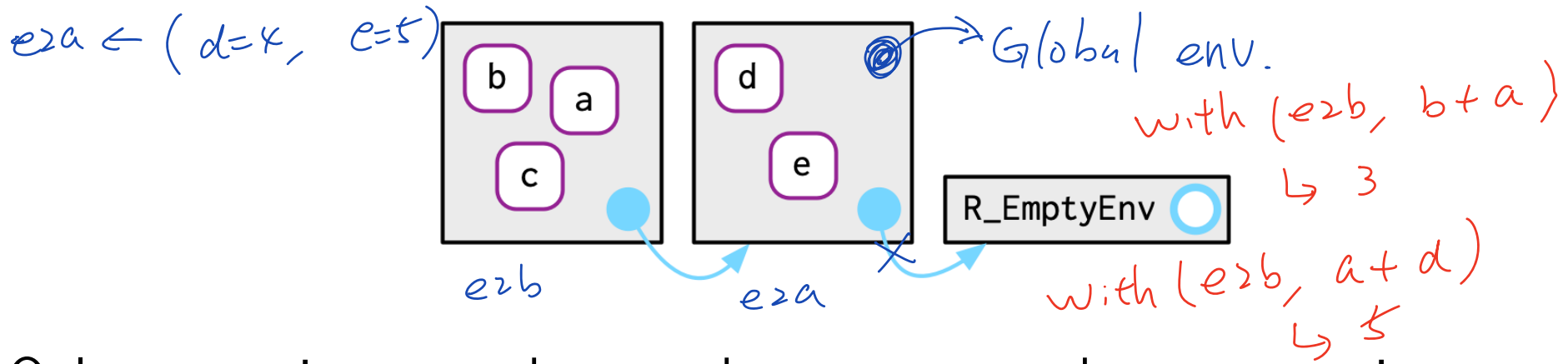
# Parent Environments

Every environment has a **parent**, another environment. In the diagrams, the pointer to the parent is a small blue circle.

The parent is used to implement **lexical scoping**: If a name is not found in an environment, then R will look in its parent (and so on).

```
e2a <- env(empty_env(), d = 4, e = 5)
e2b <- env(e2a, a = 1, b = 2, c = 3)
```

with(e2a, d+e)
↳ 9




Only one environment does not have a parent: the empty environment.

# Super Assignment

The regular assignment <- operator always creates variables within the current environment.

The **super assignment** <<- operator never creates a variable in the current environment but instead modifies an existing variable found in a parent environment.

**Warning**: If <<- does not find an existing variable, it will create one in the global environment, *even if the global environment is not the parent*. Because of this subtlety, super assignment can be tricky to use and should generally be avoided.

# Super Assignment Example

```
x <- 0
f <- function() {
  x <<- 1
}
f()
x
```

```
## [1] 1
```

# The Important Environments

There are four special environments:

- The `globalenv()`, or **global environment**, also called the **workspace**. This is the environment in which you normally work. The parent of the global environment is the last package that you attached with `library()` or `require()`.
- The `baseenv()`, or **base environment**, is the environment of the base package. Its parent is the empty environment.
- The `emptyenv()`, or **empty environment**, is the ultimate ancestor of all environments, and the only environment without a parent.
- The `environment()` is the current environment.

# Package Environments and the :: Operator

The **package environment** is the external interface to the package. This is the environment which the R user uses to find a function in an attached package. Its parent is determined by the search path, i.e. the order in which packages have been attached.

The **double colon** `::` operator can be used to access a function directly from the package environment (regardless of the search path).

For example, `base::mean()` refers to the `mean()` function from the `base` package, while `mosaic::mean()` refers to the `mean()` function from the `mosaic` package.

Double colon notation allows you to access objects that may be masked by other objects from packages higher in the search path.

# Package environments and the search path

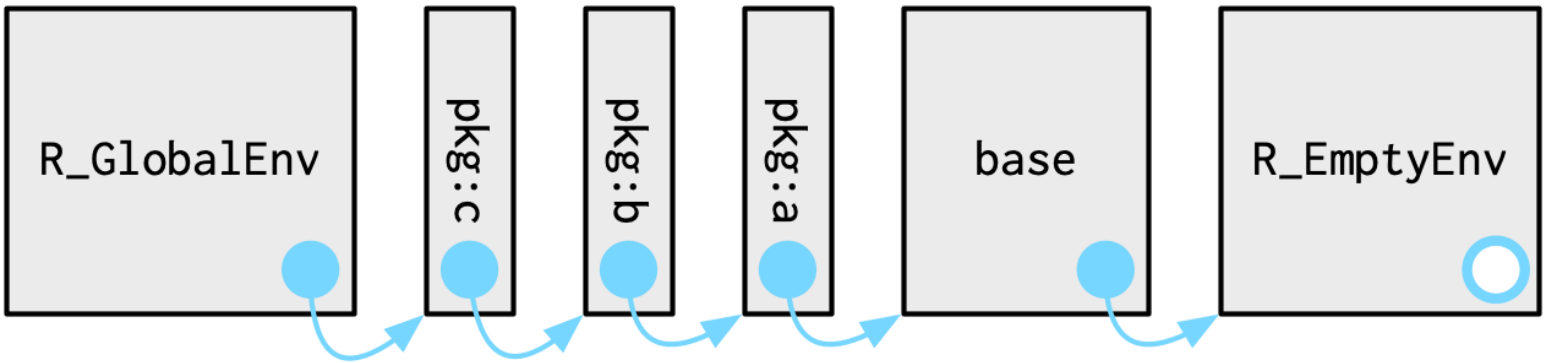
The **`search()`** function lists all parents of the global environment. This is called the **search path** because objects in these environments can be found from the top-level interactive workspace.

```
search()
```

```
##  [1] ".GlobalEnv"        "package:rlang"
##  [3] "package:knitr"     "package:stats"
##  [5] "package:graphics"  "package:grDevices"
##  [7] "package:utils"     "package:datasets"
##  [9] "package:methods"   "Autoloads"
## [11] "package:base"
```

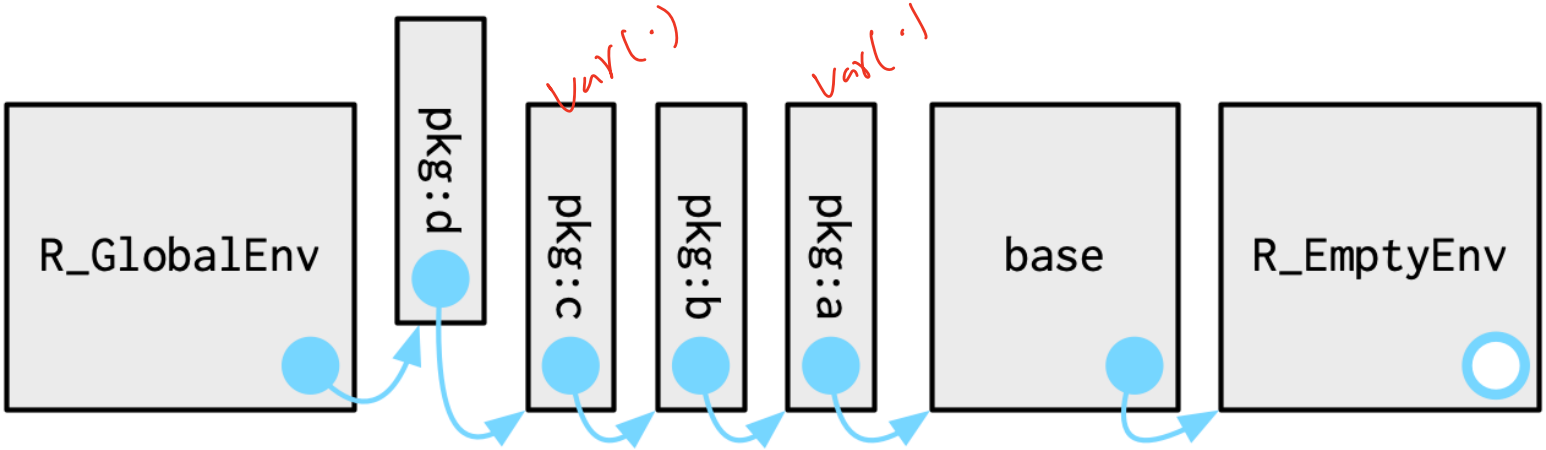
# The Search Path Visualized

The `globalenv()`, `baseenv()`, the environments on the search path, and `emptyenv()` are connected as shown below.



# The Search Path Visualized

Each time you load a new package with `library()`, the package environment is inserted between the global environment and the package that was previously at the top of the search path.




Note that the parent of the global environment changes.

# Package Environments (Cont.)

In the search path, we saw that the parent environment of a package varies based on what other packages have been loaded. Does this mean that the package will find different functions if packages are loaded in a different order?

For example, consider the `sd()` function:

```
sd = sqrt(var(x))
```

```
## function (x, na.rm = FALSE)
## sqrt(var(if (is.vector(x) || is.factor(x)) x else
as.double(x),
## na.rm = na.rm))
## <bytecode: 0x10a6e0018>
## <environment: namespace:stats>
```

The `sd()` function is defined in terms of the `var()` function. How does R make sure that the `sd()` function is not affected by any function called `var()` in the global environment or in one of the attached packages in the search path?

# Namespaces

The distinction between enclosing and binding environments is particularly important for functions inside packages.

To ensure that every package works the same way regardless of what packages are attached by the user, packages require an internal environment called the **namespace** in which the package functions are defined.

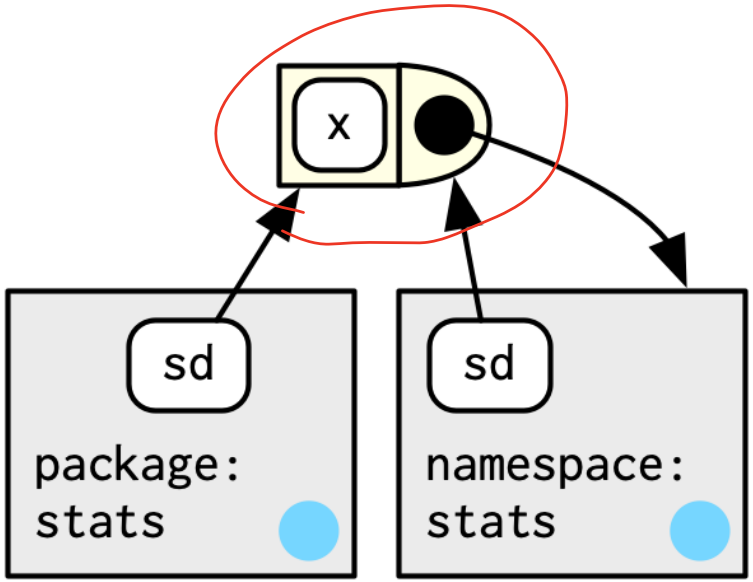
Every function in a package is associated with a pair of environments:

- the package environment and
- the namespace environment.

# Namespace Environments

The **namespace environment** is the internal interface to the package.

The package environment controls how we find the function; the namespace controls how the function finds its variables.

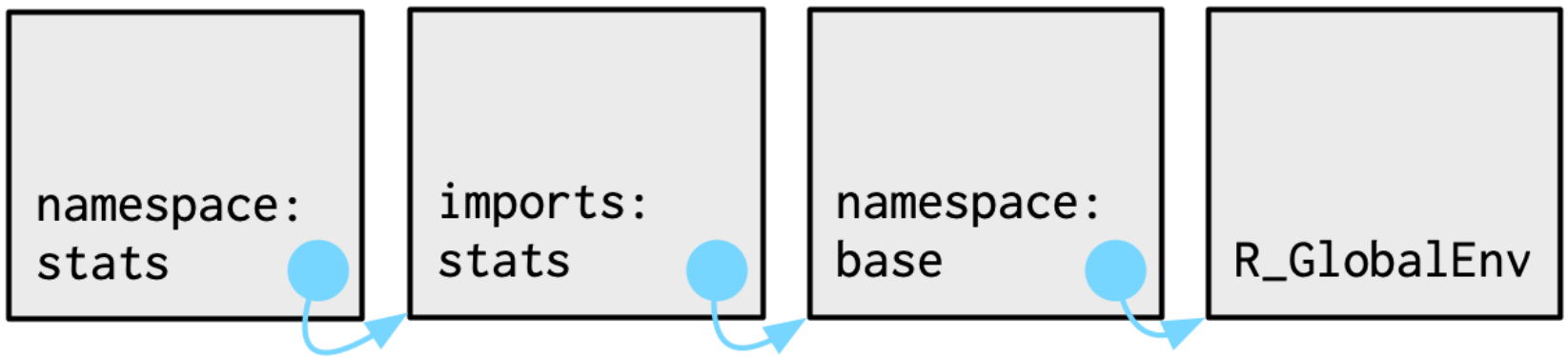



The namespace environment of a package often contains internal or non-exported variables (bindings) that allows internal implementation details to be hidden from the user.

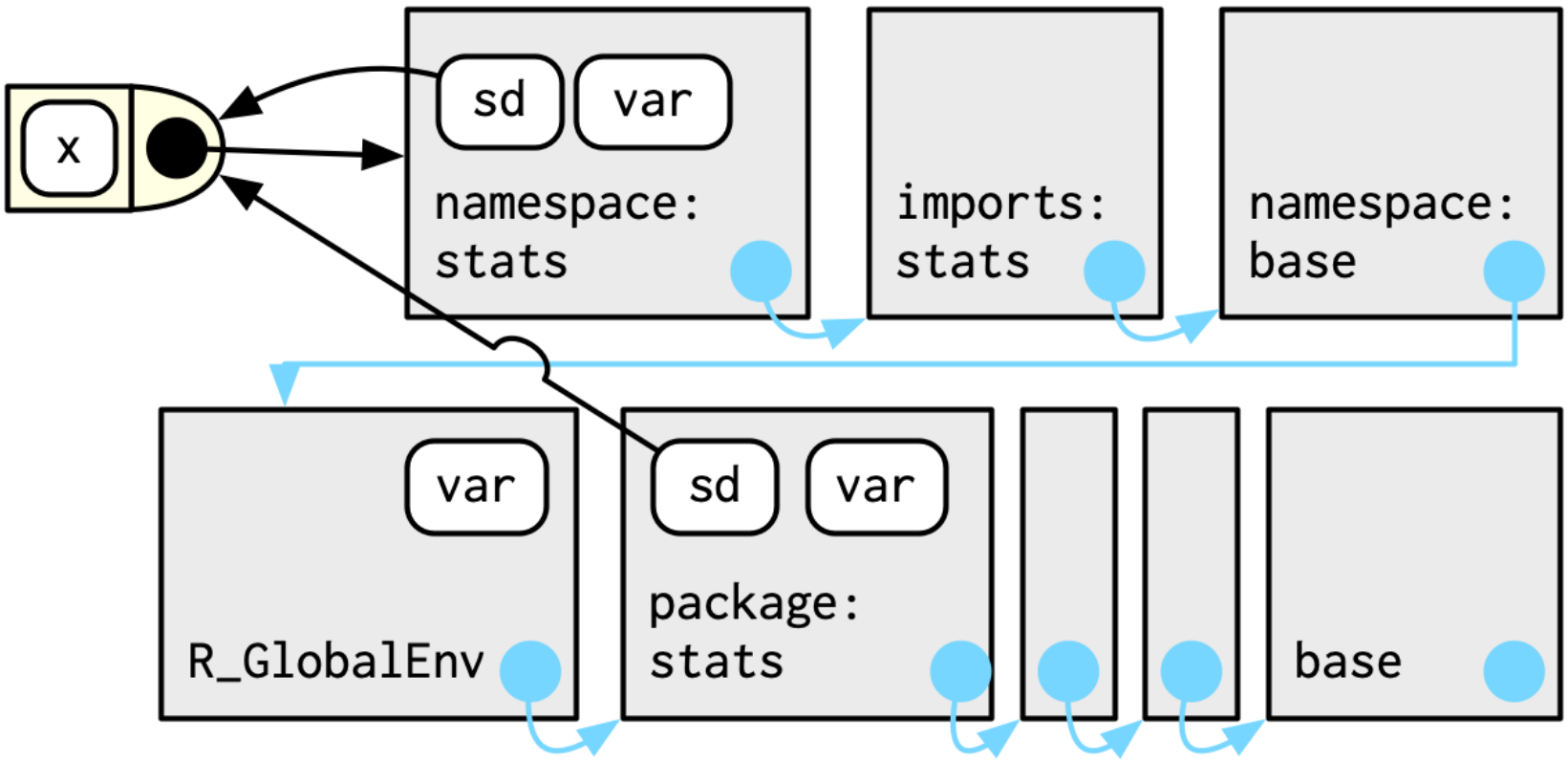
# Namespace Environments (Cont.)

Every namespace environment has the same set of ancestors:

- Imports environment contains bindings to all the functions used by the package.
- Base namespace contains the same bindings as the base environment, but it has a different parent.
- Global environment is the parent of the base namespace.



# Namespace Environments (Cont.)



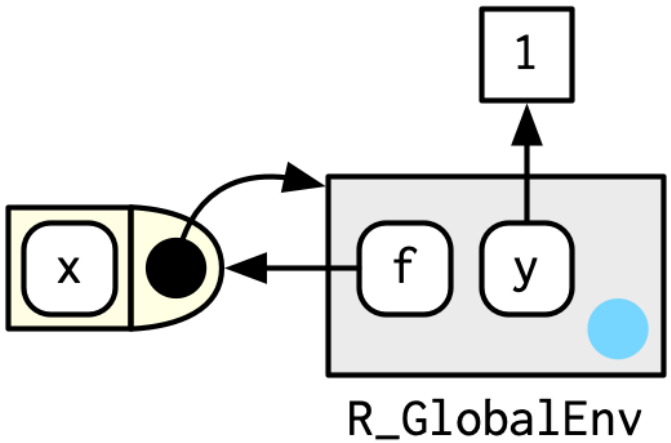
# The Function Environment

A function binds the current environment when it is created. This is called the function environment, and is used for lexical scoping. Across computer languages, functions that capture (or enclose) their environments are called **closures**.

For example:

```
y <- 1
f <- function(x){x + y}
environment(f)
```

```
## <environment: R_GlobalEnv>
```



# Execution Environments

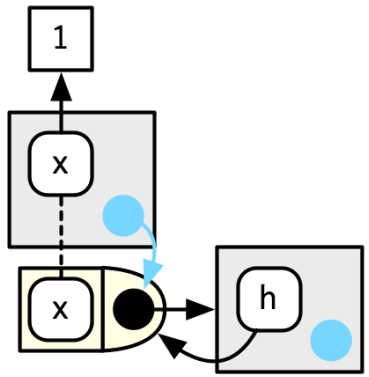
Each time a function is called, a new environment is created to host the execution. The parent of the execution environment is the function environment. Once the function has completed, this environment is thrown away.

Consider the function below:

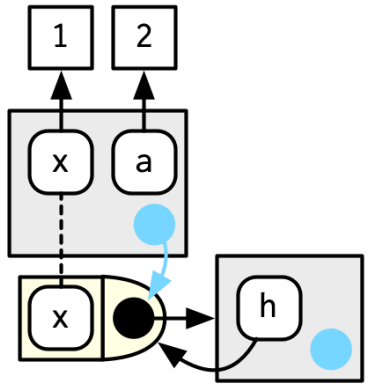
```
h <- function(x) {
  a <- 2
  x + a
}
y <- h(1)
```

# Execution Environments

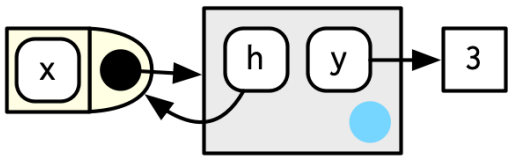
**1.** Function called with x = 1




**2.** a bound to value 2




**3.** Function completes returning value 3.
Execution environment goes away.



# Calling Environment

When you call a function, it creates an execution environment.

The execution environment has two parent environments: the enclosing environment and calling environment.

If you created a function in the `globalenv()` and call the function in the `globalenv()`, then these are the same. But in some cases, you might call a function from within another function. In this case, the enclosing environment will be the environment where the function was created, but the **calling environment** will be the environment from where it was called.

R's scoping rules will use the enclosing environment, so if the function is looking for values, it looks in the enclosing environment.

However, R also supports **dynamic scoping** where you can look for values in the calling environment.

# Call Stacks

```
f <- function() {
  g()
}
g <- function() {
  h()
}
h <- function() {
  lobstr::cst()
}
f()
```

```
##     x
##  1. \-global f()
##  2.   \-global g()
##  3.     \-global h()
##  4.       \-lobstr::cst()
```